M.Tech Program

Advanced Industry Integrated Programs

Jointly offered by University and LTI MindTree

Intelligent Cache Optimization for AI Agents

Review 2

DEV S SHAH
VIT CHENNAI
T9991

Knowledge partner

LTM

Implementation Partner

L&T EduTech

Introduction

Problem

- Repeated LLM queries cause **high cost, latency, and token wastage**
- Scalability challenges due to growing API usage and rate limits

Solution: Intelligent Caching

- Exact Match Cache : Redis
- Semantic Similarity Cache : Pgvector
- Context-Aware Retrieval : Agent/Tool support.
- **Intelligent Policy Engine**: Cost/Frequency/Recency based eviction/caching.

Objective

- **Tiered Cache Middleware**: Multi-stage interceptor for LLM/Agent calls.
- **Intelligent Storage Layer**:
  - *Exact:* Redis for deterministic lookups (low latency).
  - *Semantic:* Pgvector for context/meaning matches (prevents redundant compute).
- **Policy-Driven Optimization**: Intelligent scorer evaluates *if* result worth caching (balances cost vs. memory).
- **Observability Suite**: Streamlit dashboard tracking live hit-rates, latency, and cost savings.

Module name – Cache Engine

- **CacheEngine & DecisionEngine** — Central orchestration of the multi-level cache hierarchy (L0a–L3) with confidence-based reuse decisions, single-flight request coalescing, and controlled cache writes.
- **L0a/L0b Caching** — L0a caches deterministic function results; L0b caches embeddings using model fingerprint and text hash to avoid repeated computation.
- **L1 Exact Cache** — Redis/SQLite key-value cache using tenant, query, model, provider, prompt, context, and metadata-based composite keys.
- **L2/L3 Intelligent Caching** — L2 uses semantic similarity with Qdrant/LanceDB; L3 combines query and context vectors with metadata filtering for context-aware retrieval.
- **Invalidation & Ingestion** — InvalidationWorker synchronizes cache invalidation through Redis Streams, while Async Ingestion Manager handles background document-processing jobs with progress tracking.
- **Universal Data Loaders** — Automatically processes PDF, HTML, Office/ODF, code, CSV/JSON, and images with OCR fallback.

Methodology/Modeling Plan

- **Request Interception:** FastAPI /v1/query receives the query, context, tenant, and model information.
- **Multi-Level Lookup:** DecisionEngine checks **L0a → L0b → L1 → L2 → L3** in sequence, with timeout protection.
- **Exact & Semantic Matching:** L1 uses normalized query/context metadata, while L2 uses embeddings with cosine similarity and strict metadata filtering.
- **Context-Aware Retrieval:** L3 compares both query and context vectors and validates context metadata before returning a match.
- **Decision & Fallback:** The engine returns the first result above the confidence threshold; otherwise, single-flight coalescing ensures only one LLM generation for concurrent requests.
- **Cache Write & Invalidation:** Successful responses are conditionally written to L1 and asynchronously to L2/L3, with Redis TTL, Streams-based invalidation, and LRU eviction.

Data preprocessing & EDA

- **Synthetic Benchmark Workload:** No external dataset is required; workloads are generated deterministically at runtime.
- **Five Evaluation Types:** Paraphrase-hit, adversarial, multilingual, code-AST, and lifecycle workloads test different cache behaviours.
- **Controlled Queries:** Templates generate repeated, paraphrased, contextual, and near-miss queries with controlled entity, quarter, and topic metadata.
- **Semantic Representation:** BAAI/bge-small-en-v1.5 through FastEmbed generates **384-dimensional embeddings** for vector matching.
- **Ground Truth:** Deterministic fixtures define the expected cache result, making benchmark outcomes reproducible.
- **Lifecycle Validation:** Tests cover stale writes, refresh, invalidation, eviction, semantic matching, and multilingual/code-query scenarios.

Approach/Algorithms/Tools

Cache & Storage: Redis and SQLite support exact caching, while Qdrant and LanceDB provide vector storage for semantic and context-aware caching.
Embeddings & Similarity: FastEmbed with BAAI/bge-small-en-v1.5 performs local embedding generation and cosine-similarity search.
LLM Integration: LiteLLM provides a common interface for Gemini, OpenAI, Anthropic, Groq, and Ollama.
Application Layer: FastAPI handles asynchronous APIs, authentication, rate limiting, request IDs, and health/metrics endpoints.
UI & Observability: Streamlit provides the dashboard, while Prometheus/OpenTelemetry and structured logging support monitoring and tracing.
Deployment & Data Handling: Docker, Helm, and Kubernetes support deployment; content sniffing selects loaders using file content rather than extensions.

Project plan/milestones progress

- **Core System Completed:** L1/L2/L3 caching, metadata hard-gates, L0a/L0b caching, and the DecisionEngine hierarchy are implemented.
- **Platform Features Completed:** Multi-tenancy, API authentication, universal document loaders, asynchronous ingestion, and Redis-based distributed invalidation are operational.
- **Production Layer Completed:** FastAPI backend, monitoring, tracing, rate limiting, Streamlit dashboard, and Helm/Kubernetes deployment are implemented.
- **Benchmarking Completed:** Five benchmark datasets, automated reports, CI integration, and **0% false-hit rate on adversarial tests** have been verified.
- **Current Results:** Paraphrase benchmark achieved **88.89% cache hit rate**, avoiding **72 of 81 LLM calls**, with 72,000 tokens saved.
- **In Progress:** Accounting/security fixes, L4–L9 layers, distributed rate limiting, broader end-to-end integration testing, and Windows CI validation remain.

Pipeline structure

